Tarea 2 - Procesamiento de datos Crossref con Apache Spark
Manual de ejecucion

Descripcion

Programa en PySpark que lee los archivos JSON del dataset Crossref (articulos sobre COVID-19), los convierte a Parquet y a partir de ese Parquet genera una serie de salidas con agregaciones y metricas.

Requisitos

- Java 17, ya que Spark necesita una JVM.
- Python 3.9 o superior.
- PySpark y pytest, listados en requirements.txt.

Para instalar las dependencias de Python:

```
pip3 install -r requirements.txt
```

Si no se tiene Java instalado, se puede usar un JDK (por ejemplo Temurin 17) y apuntar JAVA_HOME hacia el antes de ejecutar:

```
export JAVA_HOME=/ruta/al/jdk-17/Contents/Home
export PATH="$JAVA_HOME/bin:$PATH"
```

Ejecucion

```
spark-submit programaestudiante.py crossref/*.json
```

El programa recibe las rutas de los archivos de entrada como argumentos. Cuando la carpeta contiene muchos archivos, el patron crossref/*.json puede superar el limite de argumentos del shell. En ese caso se puede pasar la carpeta directamente y Spark leera todos los archivos:

```
spark-submit programaestudiante.py crossref
```

Tambien funciona pasando el patron entre comillas para que lo resuelva Spark:

```
spark-submit programaestudiante.py 'crossref/*.json'
```

Los archivos son JSON con formato multilinea (un objeto por archivo), por lo que se leen con la opcion multiLine.

Salidas

Todas las salidas se escriben dentro de la carpeta output y cada una se guarda como un unico archivo usando coalesce(1):

- output/crossref-parquet: los datos originales convertidos a Parquet con el esquema reducido (doi, author, inst, groupTitle, createdDate, prefix, reference, link, subtype). El resto de operaciones lee desde este Parquet.

- output/articles_month_year: cantidad de articulos creados por anio y mes (created_year, created_month, total_articles), ordenado por anio y mes.

- output/research_group: cantidad de articulos distintos por research group (group_title, total).

- output/research_areas_per_person: por persona, la lista de research groups

distintos y su cantidad (full_name, research_areas, total), ordenado por total. El nombre vacio se reemplaza por unknown.

- output/person_most_references: la persona cuyos articulos acumulan mas referencias distintas que contienen DOI (full_name, total_references).

- output/percentil_25, output/percentil_50, output/percentil_75: el valor de total_references que corresponde a cada percentil sobre la distribucion de referencias por persona.

Pruebas

```
pytest
```

Las pruebas construyen DataFrames pequenos con el mismo esquema del Parquet y validan cada funcion del modulo crossref_processing, incluyendo casos como fechas nulas, DOIs repetidos, nombres vacios y referencias sin DOI.

Notas sobre las decisiones

- En research_group y research_areas_per_person los conteos descartan repetidos: articulos distintos por DOI en el primer caso y research groups distintos por persona en el segundo.

- En person_most_references se cuentan las referencias distintas por DOI a lo largo de todos los articulos de la persona. El valor de referencia de la tarea (Florian Krammer con 2701) coincide con este conteo. El grupo unknown (nombres vacios) no representa a una persona real, por lo que se excluye de este resultado y del calculo de percentiles.

- Los percentiles usan la funcion exacta percentile de Spark sobre la columna total_references.